Design Justification Report

Matrix Multiplication Co-Processor Chiplet

ECE 410/510 — Hardware for AI/ML, Spring 2026

Milestone 4 Final Submission

Ivan Herrera (cman) | Portland State University | June 2026

---

1. Problem and Motivation

This project accelerates dense matrix multiplication (GEMM), specifically the INT8 linear projection Y = XW that constitutes the dominant computational kernel in transformer inference workloads. Profiling a representative transformer-style linear layer (X: 128×256, W: 256×256) on the host platform — an Intel Core i7-1165G7 running Ubuntu 22.04 — confirmed that the matrix multiply routine accounted for approximately **87% of total CPU runtime**, with a median latency of 0.38 ms per call and a throughput of 44.2 GFLOPS (FP32, using PyTorch/BLAS). Memory profiling indicated an L2 cache miss rate of approximately 18% and sustained DRAM bandwidth of 14.2 GB/s.

The core limitation of the CPU implementation is that a general-purpose processor must divide its pipeline resources across many unrelated tasks and cannot dedicate its full bandwidth and compute capacity to a single sustained multiply-accumulate kernel. A custom hardware accelerator focused exclusively on GEMM can eliminate this overhead. Profiling data is documented in *project/m1/sw_baseline.md*.

2. Roofline Analysis

For a 512×512 INT8 matrix multiplication, the arithmetic intensity is computed as:

$$AI = 2N^3 \text{ FLOP} / (2N^2 \times 1 \text{ byte}) = N = 512 \text{ FLOP/byte [INT8 operands]}$$

Using the i7-1165G7 platform parameters (peak compute 44.2 GFLOPS FP32, peak DRAM bandwidth 51.2 GB/s), the ridge point is approximately 44,200 / 51,200 ≈ 0.86 FLOP/byte. At AI = 512 FLOP/byte, dense GEMM sits firmly in the compute-bound region. This means the bottleneck is multiply-accumulate throughput, not DRAM bandwidth — the primary motivation for a dedicated MAC array accelerator.

At the M4 operating point (single-PE, N=8 dot products), the system-level arithmetic intensity drops to approximately **0.24 FLOP/byte** due to the AXI4-Lite single-beat interface: each 8-bit operand pair requires two separate 4-byte AXI write transactions. This places the M4 design in the memory-bound region relative to the AXI bus ceiling. The measured throughput of 177.8 MFLOP/s (simulation) corresponds to the AXI interface bandwidth bottleneck, not the MAC datapath ceiling. See Figure 1 (roofline_final.png) for the complete roofline plot showing all operating points.

The roofline analysis directly shaped the architecture: to escape the AXI-Lite interface bottleneck and reach the compute-bound region, the design requires burst-mode AXI4 transfers and a systolic array of PEs. Both were identified as M4 targets but ultimately deferred (see Section 9).

3. Precision and Data Format

The design uses **INT8** (signed 8-bit, two's complement) for matrix operands A and B, with **INT32** (signed 32-bit) accumulation for the dot-product result. INT8 was selected over FP32, FP16, BF16, and INT4 for the following reasons:

vs FP32/FP16/BF16: Floating-point multiplication requires a multi-cycle FPU (mantissa multiply + exponent addition + normalization), adding substantial RTL complexity, synthesis area, and power. INT8 multiplication synthesizes to a compact Wallace tree adder with no normalization logic.

vs INT4: INT4 would halve data bandwidth but introduces non-trivial accuracy loss (0.5–2% top-1 drop without careful calibration) and requires additional quantization engineering outside the project scope.

Quantization error analysis (documented in *project/m2/precision.md*): Using symmetric per-tensor quantization with scale S = max|W|/127, the measured mean absolute error over 1,000 random 4×4 matrix pairs was **MAE = 0.0021**, maximum error 0.0183 — well within the 0.01 acceptability threshold from the INT8 transformer inference literature (NVIDIA TensorRT whitepaper, Q8BERT). Using an incorrect scale (S_bad = 0.01, too small by 1.8×) causes catastrophic weight clipping and raises MAE to 0.171 — approximately 40× worse. The INT32 accumulator eliminates overflow as an error source for matrix dimensions in scope.

4. Dataflow and Architecture

The M4 design implements a **no-local-reuse (streaming) dataflow**: operand pairs (a, b) are streamed in serially from the AXI4-Lite host interface, one pair per clock cycle, and accumulated in a single 32-bit register. This is not a weight-stationary or output-stationary systolic array — it is the single-PE baseline established in M2, carried forward unchanged through M3 and M4.

compute_core.sv: A single MAC engine. On each cycle where both a_valid and b_valid are asserted, the core computes product = a × b (INT8×INT8 → 16-bit signed), sign-extends to 32 bits, and accumulates into a 32-bit register. After *dim* MAC operations, result_valid pulses for one cycle and the result is presented on the result port. Latency: exactly N cycles for an N-element dot product (N = 5 measured cycles for dim=4, N = 9 for dim=8 — confirmed in simulation).

axi_slave.sv (formerly interface.sv): AXI4-Lite slave exposing five memory-mapped registers: CTRL (0x00), A_DATA (0x04), B_DATA (0x08), STATUS (0x0C), RESULT (0x10). The host writes DIM and START to CTRL, streams element pairs via A_DATA and B_DATA writes, polls STATUS.RESULT_VALID, then reads the RESULT register.

top.sv glue logic: The AXI slave issues a_valid and b_valid as independent one-cycle pulses (one per AXI write transaction). The compute_core requires both valid signals simultaneously. A two-register synchronization latch in top.sv holds whichever element arrives first and fires the joint pulse when the second arrives. This adds at most one cycle of latency per pair and is the only glue logic required between the two modules.

5. Hardware Interface

The selected interface is **AXI4-Lite** (slave, 8-bit address, 32-bit data). AXI4-Lite was selected over SPI, I2C, and full AXI4 for M2/M3/M4 because it is the standard control interface for ASIC/FPGA memory-mapped peripherals, is on the grader-approved allowed list, and provides adequate bandwidth for the single-PE operating point.

Effective bandwidth: At 100 MHz with a 4-byte write data bus, AXI4-Lite sustains approximately 400 MB/s peak (one 4-byte transaction per clock cycle at full utilization). For N=8, streaming 16 operand bytes requires a minimum of 16 cycles at peak throughput. The required bandwidth to sustain 177.8 MFLOP/s at AI = 0.24 FLOP/byte is 177.8 / 0.24 ≈ **740 MB/s**. The AXI4-Lite interface at 100 MHz (400 MB/s peak) is therefore the binding constraint: the design is **interface-bound** at the system level.

Quantification: The MAC datapath itself runs at 100 MHz × 2 FLOP/cycle = 200 MFLOP/s theoretical peak. The measured 177.8 MFLOP/s represents 89% of the MAC datapath ceiling, but only 0.024% of the AXI4-Lite rated bandwidth, confirming that the interface — not the compute — is the bottleneck. Upgrading to burst-mode AXI4 (as planned for M4 but deferred) would increase AI and shift the design toward its compute ceiling. See *project/m1/interface_selection.md* for the original bandwidth requirement analysis.

6. Verification

Functional correctness was verified at three levels of integration across M2, M3, and M4, all using Icarus Verilog 12.0 (iverilog -g2012).

M2: Unit-level testbenches

tb_compute_core.sv: Drives the MAC kernel directly with test vector A=[3,–1,2,5], B=[4,7,–3,1], dim=4. Expected result: 12–7–6+5 = 4. Verified independently in NumPy. Log: *project/m2/sim/compute_core_run.log* — result: PASS.

tb_interface.sv: Verifies AXI4-Lite write/read transactions, CTRL register decoding (DIM=4, START=1), A_DATA/B_DATA routing, and RESULT register read-back with injected core_result = 42. Log: *project/m2/sim/interface_run.log* — all 5 checks: PASS.

M3: End-to-end co-simulation

tb_top.sv: Full AXI4-Lite transaction sequence through the integrated top module. Phases: CTRL write → N element pair writes (A_DATA, B_DATA) → STATUS poll → RESULT read. Same test vector as M2. Log: *project/m3/sim/cosim_run.log* — result: PASS.

M4: Final two-test verification

project/m4/tb/tb_top.sv: Two back-to-back end-to-end tests with full reset between them. Test 1: A=[3,–1,2,5]·B=[4,7,–3,1] = 4 (mixed signs). Test 2: A=[–2,–3,–1,–4]·B=[1,2,3,4] = –27 (all-negative A). Log: *project/m4/sim/final_run.log* — both tests: PASS. Waveform: *project/m4/sim/final_waveform.png*.

The *interface* keyword conflict with Icarus Verilog 12 and Yosys is a known issue documented throughout the project README files. The module was renamed to *axi_slave* in all M4 deliverables as the standard workaround.

7. Synthesis Results

Synthesis was performed with Yosys 0.33 (git sha1 2584903a060) targeting the generic ABC genlib library (sky130_fd_sc_hd OpenLane 2 run was not available in the build environment; config.json is committed and ready for an OpenLane 2 host).

Module	Cells	Flip-Flops	Notes
top (glue logic)	59	35	A/B sync latch + port wiring
axi_slave (interface)	191	111	AXI4-Lite slave, 5-register map
compute_core	1,849	70	INT8×INT8→INT32 MAC, FSM, adder tree
TOTAL (flat hierarchy)	2,097	216	0 problems, no latches, no loops

Area: The dominant area contributor is the compute_core INT8×INT8→INT32 multiplier (525 XOR + 579 ANDNOT cells, ~53% of all logic cells). Estimated sky130 area: 2,097 cells × 2 µm²/cell ≈ 4,194 µm², or approximately 10.5% core utilization of the 200×200 µm die configured in config.json.

Timing: The critical path runs from the accumulator register through the 32-bit multiplier (~2.8 ns) and 32-bit ripple-carry adder (~3.2 ns), with estimated total combinational delay of ~6.8 ns. Estimated worst-case slack at 100 MHz: +3.2 ns (positive — timing expected to close). See *project/m4/synth/timing_report.txt*.

Power: Full switching-activity power estimation requires OpenLane 2 / OpenROAD with sky130 liberty arcs (not available in the build environment). Cell-count estimate: leakage ~2.1 µW + dynamic ~135 nW ≈ **~2.2 µW total** (excluding routing parasitics, which typically add 1.5–3× to dynamic power). See *project/m4/synth/power_report.txt* for the full explanation of what was attempted and why a tool-generated report is unavailable.

8. Benchmark Results

All hardware numbers are **simulation-measured** via cycle-accurate RTL simulation (Icarus Verilog 12.0). The measurement method is clock cycles from start assertion to result_valid — identical to the method used in M2 and CF09. Raw data: *project/m4/bench/benchmark_data.csv*.

Metric	SW Baseline (INT8)	HW Accelerator	Speedup
N=8 latency	1,121 ns	90 ns (sim-measured)	12.5×
N=8 throughput	14.27 MFLOP/s	177.8 MFLOP/s (sim-measured)	12.5×
N=4 latency	1,142 ns	50 ns (sim-measured)	22.8×
N=4 throughput	7.00 MFLOP/s	160.0 MFLOP/s (sim-measured)	22.9×
8×8 matmul latency	71.7 µs (est.)	6.39 µs (sim-measured)	11.2×
Memory footprint	~329 KB (NumPy runtime)	~7 bytes (registers)	~47,000×

The measured 12.5× speedup (N=8) comes from eliminating Python/NumPy interpreter overhead, cache miss penalties, and general-purpose CPU pipeline stalls. The hardware accelerator runs the MAC datapath at 100 MHz with zero Python overhead and a ~7-byte register footprint versus 329 KB of NumPy runtime state.

Gap between measured and theoretical performance: The Heilmeier Q3 target is 55–65 GFLOPS, achieved via a multi-PE systolic array. The M4 single-PE design achieves ~178 MFLOP/s — approximately 330–365× below the target. This gap is entirely explained by the single-PE architecture: a 16×4 systolic array (64 PEs) operating at 100 MHz would achieve approximately 64 × 200 MFLOP/s = 12.8 GFLOP/s, and a 256-PE array would reach the 51 GFLOP/s range. The AXI4-Lite interface would need to be replaced with burst AXI4 to sustain that throughput.

Energy: Estimated accelerator energy per N=8 dot product: ~2.2 µW × 90 ns ≈ 0.2 pJ (cell-count estimate, excluding routing parasitics). CPU equivalent: ~3 W × 1,121 ns ≈ 3.4 µJ — approximately **17,000× more energy per operation**. Even at 10× the estimated power (~22 µW), the hardware remains ~1,700× more energy-efficient. These are order-of-magnitude estimates, not measurements.

9. What Did Not Work

This section documents specific failures, setbacks, and scope reductions encountered across the project.

9.1 Systolic array and SRAM buffers not implemented

The Heilmeier Q3 target — a systolic-array compute core with on-chip SRAM tile buffers targeting 55–65 GFLOPS at sky130 — was not implemented. This was the largest single scope gap. The remaining_tasks.md file committed in CF09 identified three pre-M4 tasks: (1) implement a 4×4 systolic PE array with SRAM tile buffer, (2) run OpenSTA post-synthesis timing, (3) replace AXI4-Lite with burst AXI4. None of the three were completed. The core reason was underestimating the integration complexity of the A/B synchronisation latch in top.sv — debugging the joint-valid glue logic consumed significant time that was planned for systolic array development. In hindsight, the latch should have been designed and tested in isolation before the M3 integration step.

9.2 iverilog / Yosys 'interface' keyword conflict

Naming the AXI slave module 'interface' conflicted with the SystemVerilog 'interface' reserved keyword in Icarus Verilog 12 and certain Yosys parse paths. Symptom: iverilog would reject the top-level instantiation with a cryptic error rather than a clear 'reserved keyword' message. Resolution: rename to 'axi_slave' throughout. This cost approximately half a day of debugging in M2 before the root cause was identified. The lesson: avoid SystemVerilog reserved words as module names even when a particular tool version accepts them, because downstream tools may not.

9.3 @(posedge result_valid) race condition in testbench

The initial M2 testbench used '@(posedge result_valid)' to wait for the core output. This created a race condition in Icarus Verilog: if result_valid was already high when the event monitor was registered (possible if the clock edge and the $display happened within the same simulation time step), the testbench would miss the pulse and hang. Resolution: switched to a flat polling loop — sample result_valid on every posedge clk — which is more robust in event-driven simulators. The M3 and M4 testbenches both use the polling approach.

9.4 OpenLane 2 / OpenSTA not available in build environment

The M4 synthesis reports are based on Yosys 0.33 with the generic ABC genlib library, not a full OpenLane 2 / sky130 PDK run. OpenLane 2 requires a Linux host with Docker, approximately 25 GB of PDK files, and a multi-hour install process. The build environment used for M4 (this container) does not have Docker or the sky130 PDK installed. As a result, the timing and power reports are estimates rather than tool-generated numbers, and no physical-level area report (in µm²) is available from the PnR tool. The config.json is committed and valid; the openlane_run.log contains the full Yosys synthesis log as a substitute. The power section explicitly documents this limitation per the M4 checklist requirement.

9.5 AXI4-Lite interface bottleneck identified too late

The arithmetic intensity analysis in CF09 revealed that the AXI4-Lite single-beat interface limits system-level AI to ~0.24 FLOP/byte, placing the design in the memory-bound region of the roofline. This was known in principle from M1 (interface_selection.md noted that AXI4-Lite is for control, not bulk data), but the quantitative impact on the benchmark numbers was not fully understood until CF09. Replacing AXI4-Lite with burst AXI4 was identified as a required change but was not implemented. If this project were repeated, burst AXI4 would be designed in from the start of M2.

Figures

Figure 1: Final roofline plot — project/m4/bench/roofline_final.png. Shows the i7-1165G7 and sky130 single-PE roofline ceilings, the M1 FP32 GEMM software baseline, the INT8 software baseline, the M4 hardware accelerator (simulation-measured at 177.8 MFLOP/s, AI=0.24 FLOP/byte), and the Heilmeier projected target (not built).

Figure 2: Final waveform — project/m4/sim/final_waveform.png. Shows AXI4-Lite signal activity across both M4 test cases: AWVALID, AWREADY, WVALID, WREADY, BVALID, ARVALID, RVALID annotated with PASS markers.

Block diagram and dataflow diagram are described textually in Sections 4 and 5; the systolic array dataflow is documented in codefest/cf05/cman_systolic_trace.md.

Word count (approximate): 2,850 words — within the 2,000–5,000 word requirement.